

Code review Assad (v2)

☰ Type	
↗ Project tags	<u>Curator program</u>
↗ Users	<u>Ahmed</u>
⌚ Created by	Ⓐ Armando
⌚ Created time	March 13, 2026 3:53 AM
⌚ Last edited by	Ⓡ Rushab Taneja
⌚ Last edited time	March 31, 2026 7:12 PM
↗ Authors	🇵🇪 <u>Armando</u>

▼ Rushab review

Asad **completely rewrote** the codebase. The v1 single-file script is now a proper **config-driven framework** with a separate engine (`engine.ts`), validation module (`validate.ts`), typed config schema (`config.ts`), and 34 passing tests. The SDK was upgraded from ^0.7.0 to ^0.13.0. Data files are included with 207 smart contracts, 122 audit reports, 44 protocols, 44 auditors, and license entities. The architecture is clean and well-separated.

The code **compiles cleanly** and all **34 tests pass**. The engine produces correct SDK calls with proper typed values. This is the most significant improvement among all the re-reviewed codebases.

v1 Issue	Status	Notes
Wrong type IDs (root-space vs space-specific)	FIXED	All IDs now correct (verified via API)
Wrong property IDs	FIXED	Custom properties with pre-generated IDs
No data files	FIXED	207 contracts, 122 audits, 44 protocols, 44 auditors, 3 licenses

v1 Issue	Status	Notes
<code>queryPropertyByName</code> / <code>queryTypeByName</code> identical to <code>queryEntityByName</code>	N/A	Complete rewrite — engine doesn't do runtime property/type lookup, uses config-declared IDs
Relation target entities created with no types	FIXED	Types assigned via config (<code>"type": "company"</code> , <code>"type": "project"</code>)
No validation	FIXED	Full validation module with address/URL/date/cross-reference checks
No tests	FIXED	34 passing vitest tests covering engine and validation
SDK ^0.7.0	FIXED	Upgraded to ^0.13.0
No dry-run mode	Partial	No <code>--dry-run</code> flag, but ops are saved to file before publish

Few Fixes to do to get the publishing correct -

1. `blockNumber` values are string floats — integer encoding will receive wrong type

File: `data_to_publish/smart_contracts.json`

All 207 `blockNumber` values are **strings with decimal points**: `"16291130.0"`, `"12504268.0"`, etc. The config declares `blockNumber` as `"dataType": "INTEGER"`, so `VALUE_TYPE_MAP` maps it to `"int64"`.

The engine's `buildValues` passes the raw JSON value directly: `{ property: propId, type: "int64", value: "16291130.0" }`. The GRC-20 encoder's `writeSignedVarint` expects a JavaScript `number`, not a string. Depending on the SDK's internal coercion, this either:

- Silently produces wrong values (string coerced to NaN)
- Throws at encode time

Additionally, 66 contracts have `blockNumber: ""` (empty string). The engine checks `if (val == null) continue` (line 454), but `""` is not null — it passes through as `{ type: "int64", value: "" }`.

Suggested fix: Add coercion in `buildValues`:

```
if (valueType === "int64" || valueType === "float64") {
  const num = Number(val);
  if (isNaN(num) || val === "") continue; // skip invalid
  numbers
  values.push({ property: propId, type: valueType, value:
Math.floor(num) });
} else {
  values.push({ property: propId, type: valueType, value:
val });
}
```

2. "Ethereum Name Service" protocol reference doesn't resolve

Files: `data_to_publish/smart_contracts.json`, `bounty.config.json`

3 smart contracts (ENSRegistry, ENSBaseRegistrar, ENSNameWrapper) reference `"protocol": "Ethereum Name Service"`. But `existingEntities.protocols` has the entry as `"ENS (Ethereum Name Service)"` — a different key.

In `buildRelations`, the lookup `sourceMap.get("Ethereum Name Service")` returns `undefined`, so the protocol relation is silently dropped for all 3 ENS contracts.

Suggested fix: Either rename the key in `existingEntities` to `"Ethereum Name Service"`, or change the data to `"ENS (Ethereum Name Service)"`.

3. "Competitive audit contest" audit type not in existingEntities

Files: `data_to_publish/audit_reports.json`, `bounty.config.json`

5 audit reports reference `"auditType": "Competitive audit contest"`, but `existingEntities.auditTypes` only has `"Smart Contract Audit"`. The relation lookup fails silently — these 5 audits lose their audit type classification.

Affected: Ajna/Sherlock, Panoptic/Code4rena, Trader Joe/Code4rena, Lens Protocol/Code4rena, Premia/Sherlock.

Suggested fix: Add `"Competitive audit contest": "<entity_id>"` to `existingEntities.auditTypes`, creating the entity first if it doesn't exist.

4. No `-dry-run` CLI flag

File: `publish.ts`

The script validates, builds ops, and immediately publishes. Ops are saved to `data_to_delete/publish_ops.json` before publishing (line 84), but there's no way to preview without sending a transaction. A `--dry-run` flag that stops after the build summary would be valuable.

5. `web_url` property ID resolves to "Website" not canonical "Web URL"

File: `src/constants.ts:35`

`eed38e74...` → "Website" Property entity. The canonical "Web URL" property used in the Geo ecosystem is `412ff593e9154012a43d4c27ec5c68b6`. Both are valid Property entities, but using a different one means protocol URLs won't group with other entities' URLs.

6. `.DS_Store` committed to git (private files should not be committed, this time its a metadata file, next time it could be private key, so preventing this should be first priority always)

File: `.DS_Store`

macOS metadata file. Should be in `.gitignore`.

Asad went from a broken single-file script with wrong IDs and no data to a **properly-architected config-driven framework** with validation, tests, and rich dataset. The code compiles cleanly, all 34 tests pass, and the architecture is genuinely reusable.

0 CRITICAL issues. The HIGH issues are data quality problems (string float block numbers, one name mismatch), not code bugs. The framework itself is sound — it correctly creates properties, types, enums, entities with typed values, and multi-source relations.

With the data fixes applied (convert block numbers to integers, fix ENS name, add "Competitive audit contest" type), this would be ready to publish 207 smart contracts, 122 audit reports, 44 protocols, and 44 auditors to the knowledge graph — covering 97 of the required 100 protocols.